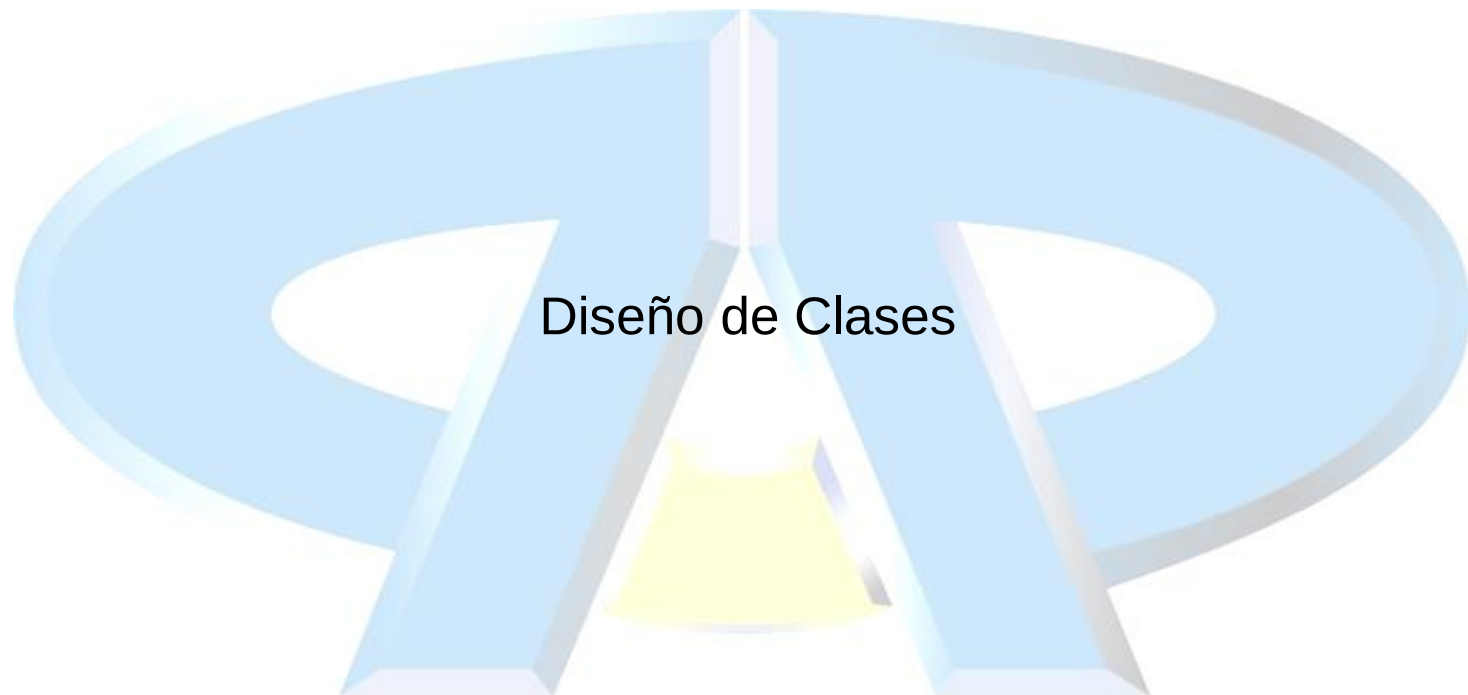




Objetivos

- Definir los conceptos de:
 - Herencia
 - Polimorfismo
 - Sobrecarga
 - Sobreescritura
 - Invocación virtual de métodos
- Utilizar los modificadores **protected** y **default** (package friendly)
- Describir los conceptos de **constructor** y sobrecarga de métodos
- Describir el proceso completo de construcción de un objeto y las operaciones de inicialización
- Los conceptos a estudiar en este capítulo son aplicables tanto en el concreto de ámbito de Java como en el de Programación Orientada a Objetos

Subclases

- Herencia es una característica de la POO que permite, a partir de un modelo genérico, crear una versión especializada
- Por ejemplo, en una empresa todos sus empleados pueden ser modelados con una clase “*Employee*” en donde se recogen todas las características (atributos) y funcionalidades (métodos) comunes a todos ellos
 - Sin embargo, determinadas características y funcionalidades pueden ser específicas de, solamente, determinados tipos de empleados
 - Por ejemplo, en una empresa puede haber empleados de diferentes tipos como:
 - Ingenieros (*Engineer*)
 - Administradores (*Manager*)
 - Secretarios (*Secretary*)

Subclasses

```
class Employee {  
    private String name;  
    private MyDate birthDate;  
    private float salary;  
    public String getDetails() {...}  
}
```

```
class Manager {  
    private String name;  
    private MyDate birthDate;  
    private float salary;  
    private String department;  
    public String getDetails() {...}  
}
```

Subclases

- Condición: Para considerar que un clase puede ser una subclase de otra más genérica, debe cumplir la siguiente afirmación:
 - La “*clase hija*” es un/a “*clase padre*”
 - En el ejemplo que se comenta, un administrador es un empleado; por lo tanto, “Manager” puede ser un subclase de “Employee”
- Implementación de herencia en Java:

```
class Manager extends Employee {  
    private String department;  
}
```

Herencia Simple

- El lenguaje de programación Java solamente permite que una clase “extienda” o “herede” de otra clase, nunca de más de una.
 - Esta restricción se denomina “herencia simple”
 - Otros lenguajes de programación permiten que una clase herede atributos y funciones de varios “padres”, lo que se denomina “herencia múltiple”
 - En Java existe un elemento que intenta suplir la herencia múltiple sin los inconvenientes que ésta tiene (*interfaces*)

Control de Acceso

- Las variables y los métodos pueden tener uno de los siguientes cuatro niveles de acceso: private, default, protected o public
- Criterios de accesibilidad:

Modificador	Misma Clase	Mismo paquete	Subclase	Universal
private	Sí			
default	Sí	Sí		
protected	Sí	Sí	Sí	
public	Sí	Sí	Sí	Sí

Clases sealed

- En la versión 17 se han añadido las clases sealed, que permiten controlar qué clases hija puede tener una clase, limitando de esta forma la herencia:

```
public sealed class Persona permits Alumno, Profesor {  
    ...  
}
```

- La clase Persona solamente permite tener dos clases hijas, Alumno y Profesor.
- Obligatoriamente, las clases hijas tienen que ser final, sealed o non-sealed

```
public final class Alumno extends Persona {  
    ...  
}  
public non-sealed class Profesor extends Persona {  
    ...  
}
```


Sobrescritura de métodos

- Además de crear nuevas clases a partir de otras preexistentes, añadiéndoles a éstas nuevos métodos, también se puede modificar el comportamiento de la clase madre, “sobrescribiendo” alguno de sus métodos
- Si un método es definido en una subclase con el mismo nombre, tipo de retorno y lista de argumentos que un método definido en la clase madre, se dice que el método de la clase madre es “sobrescrito” por el de la clase hija
 - A partir de la versión J2SE 5.0 el tipo de retorno en la clase hija puede ser una subclase del tipo de retorno definido en la clase madre

Sobrescritura de métodos

```
class Employee {  
    private String name;  
    private MyDate birthDate;  
    private float salary;  
  
    public Employee(String name, MyDate DoB, float salary) {  
        this.name = name;  
        this.birthDate = DoB;  
        this.salary = salary;  
    }  
    public String getDetails() {  
        return "Name: " + name + "\nSalary: " + salary  
            + "\nBirth Date: " + birthDate;  
    }  
}
```

Sobrescritura de métodos

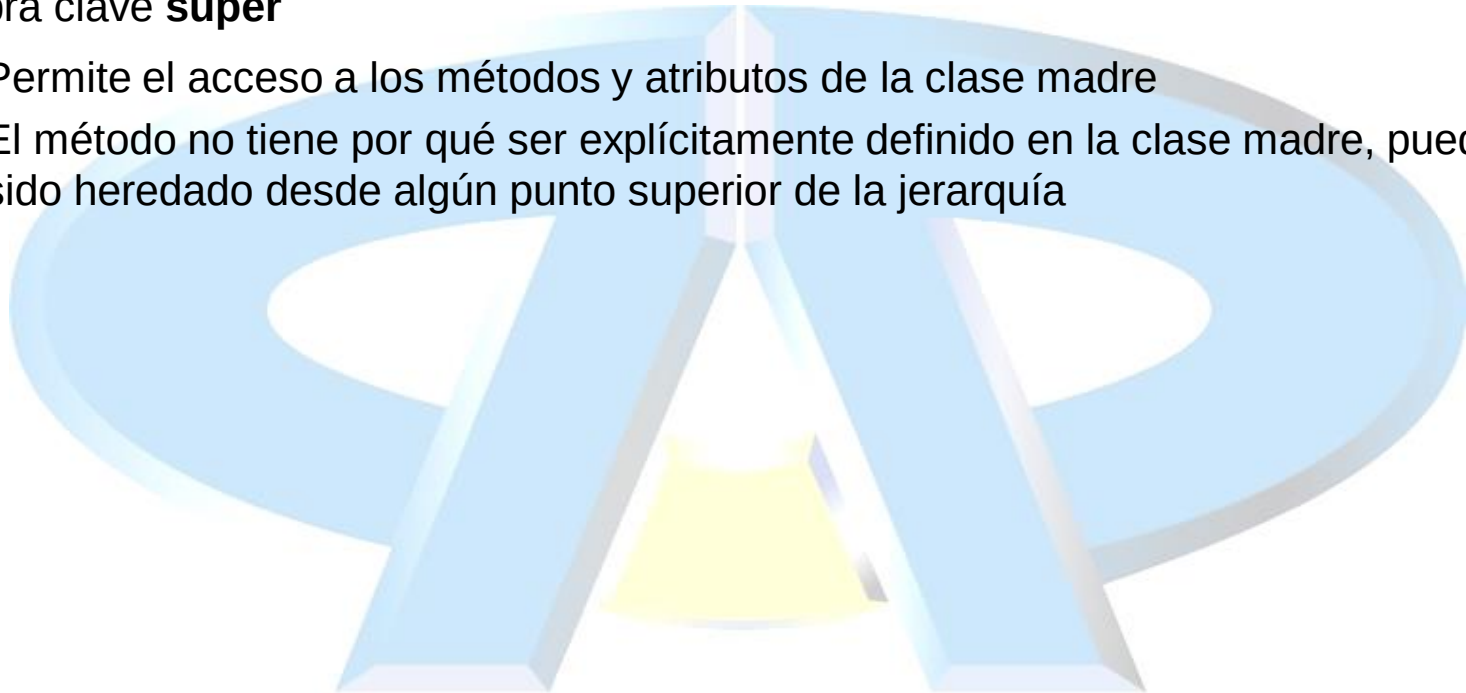
```
class Manager extends Employee {  
    private String department;  
    public Manager(String name, MyDate DoB, float salary, String dept) {  
        super(name, DoB, salary);  
        this.department = dept;  
    }  
    public String getDetail() {  
        return super.getDetails() + "\nDepartment: " + department;  
    }  
}
```

La sobrescritura de métodos no puede ser menos accesible

```
public class Parent {  
    public void doSomething() {}  
}  
  
public class Child extends Parent {  
    private void doSomething(){} // Illegal  
}  
  
public class UseBoth {  
    public void doOtherThing();  
    Parent p1 = new Parent();  
    Parent p2 = new Child();  
    p1.doSomething();  
    p2.doSomething(); // Situación de error  
}
```

Invocación de Métodos Sobrescritos

- Un método de una subclase puede invocar a métodos de su superclase utilizando la palabra clave **super**
 - Permite el acceso a los métodos y atributos de la clase madre
 - El método no tiene por qué ser explícitamente definido en la clase madre, puede haber sido heredado desde algún punto superior de la jerarquía



Sobrescritura de métodos

```
class Employee {  
    private String name;  
    private MyDate birthDate;  
    private float salary;  
  
    public Employee(String name, MyDate DoB, float salary) {  
        this.name = name;  
        this.birthDate = DoB;  
        this.salary = salary;  
    }  
  
    public String getDetails() {  
        return "Name: " + name + "\nSalary: " + salary  
            + "\nBirth Date: " + birthDate;  
    }  
}
```

Sobrescritura de métodos

```
class Manager extends Employee {  
    private String department;  
    public Manager(String name, MyDate DoB, float salary, String dept) {  
        super(name, DoB, salary);  
        this.department = dept;  
    }  
    public String getDetails() {  
        return super.getDetails() + "\nDepartment: " + department;  
    }  
}
```

Polimorfismo

- Un **objeto** tiene una **única** forma, que es la del constructor desde el que ha sido creado
- Una variable de **referencia** o puntero es **polimórfica** puesto que puede referirse a diferentes tipos de objetos
 - En Java para referenciar a un objeto puede utilizarse una variable del tipo al que pertenece el objeto o cualquier otro tipo de variable de referencia que sea del tipo de cualquier superior jerárquico en la ruta de herencia
 - Por lo tanto: `Employee e = new Manager(); // Completamente legal`
 - Sin embargo utilizando la variable **e** solamente se tiene visibilidad de los atributos y métodos que son parte de *Employee*. Los pertenecientes a *Manager* específicamente permanecen ocultos
 - Por lo tanto: `e.department = "Ventas" // Ilegal`

Invocación Virtual de Métodos

- Supóngase el siguiente escenario:

```
Employee e = new Employee();
```

```
Manager m = new Manager();
```

- Está claro que si se invocan los métodos `e.getDetails()` y `m.getDetails()` se están invocando métodos diferentes, con comportamientos diferentes
- Sin embargo, supóngase otro escenario:

```
Employee e = new Manager();
```
- El comportamiento en este caso está **asociado con el tipo de objeto** instanciado
 - Este comportamiento se denomina Invocación Virtual de Métodos
 - Justificación del problema de legalidad asociado a las restricciones de métodos sobrescritos

Colecciones heterogéneas

- Pueden crearse colecciones de objetos del mismo tipo, que son denominadas colecciones homogéneas
 - Arrays de Accounts o de Customers en los ejercicios de clase
- En Java existe una clase **Object** que permite construir colecciones de cualquier tipo de elemento, debido a que todas las clases extienden a la clase **Object**
 - Reciben el nombre de colecciones heterogéneas
 - Para incluir tipos primitivos en estas colecciones hay que hacer uso de las “Wrapper Classes”
- Ejemplo:
 - `Employee[] staff = new Employee[1024];`
 - `staff[0] = new Employee(...);`
 - `staff[1] = new Engineer(...);`
 - `staff[2] = new Manager(...);`

Argumentos Polimórficos

- Pueden crearse métodos que reciban como parámetro, en su definición un objeto genérico (e.g. Employee), y que al invocarlo pasando como parámetro un objeto subclase de la que existe en la definición (e.g. Manager), funcionen adecuadamente.
 - Esto se debe a que los parámetros que se utilizan son variables de referencia y, por lo tanto, son polimórficos (igual que cualquier otra variable de referencia)
 - Sin embargo, es necesario recordar que solamente estarán accesibles los métodos y atributos definidos en la clase genérica

```
public class TaxService {  
    public class TaxRate findTaxRate(Employee e) {  
        //Realiza cálculos y devuelve un porcentaje de retención asociado al  
        //salario  
    }  
}  
  
TaxService taxSvc = new TaxService();  
Manager m = new Manager();  
TaxRate t = taxSvc.findTaxRate(m);
```

Operador **instanceof**

- Debido a que se pueden manejar objetos de clases específicas (Manager, Director, ...) utilizando variables de referencia de clases más generales (Employee), puede suceder que en algunos casos, al procesar un objeto o una colección de objetos, se desconozca, a priori, el tipo específico de cada uno de los objetos.
- Por ejemplo:

```
public class Employee extends Object;
public class Manager extends Employee;
public class Engineer extends Employee;
public void doSomething(Employee e) {
    if (e instanceof Manager) {
        // Procesar e como un Manager
    }
    else if (e instanceof Engineer) {
        // Procesar e como un Engineer
    }
}
```

Operador **instanceof**

- A partir de la Versión 16, se añade el Pattern Matching para INSTANCEOF, permitiendo realizar la conversión de tipos en el propio operador:
 - Antes:

```
Object obj = "PRUEBA";  
if (obj instanceof String) {  
    String t = (String) obj;  
    // resto del código...  
}
```
 - Ahora:

```
Object obj = "PRUEBA";  
if (obj instanceof String t) {  
    // resto del código...  
}
```

Casting de Objetos

- Cuando se recibe una referencia a una clase padre (Employee) y se ha determinado que el objeto es de una subclase particular, utilizando para ello el operador **instanceof**, se puede acceder a la funcionalidad completa del objeto utilizando una referencia de la propia subclase, accediendo así a toda la funcionalidad del objeto.

```
public void doSomething(Employee e) {  
    If (e instanceof Manager) {  
        Manager m = (Manager) e;  
        System.out.println("Este es el director del departamento " +  
            m.getDepartment());  
        // Resto del proceso  
    }  
}
```

Casting de Objetos

- El “casting” o conversión de objetos está sujeta a una serie de comprobaciones:
 - Casts upward: Conversión hacia arriba en la jerarquía de clases
 - Siempre está permitida
 - No requiere la utilización de instanceof
 - Se permite la asignación directa
 - Casts downward:
 - El compilador comprueba que la conversión es válida
 - Un “Manager” es un “Employee”
 - Un “Manager” NO es un “Engineer” → Error
 - Si el compilador permite el cast, se comprueba la naturaleza del objeto en tiempo de ejecución
 - Si la conversión es correcta → OK
 - Si los objetos son incompatibles → Excepción (Error en tiempo de ejecución)

Sobrecarga de métodos

- Métodos de una clase que, básicamente, realizan la misma funcionalidad pero reciben una lista diferente de parámetros.
 - Se puede reutilizar el nombre de un método en una clase, con parámetros diferentes
 - Cuando se invoca el método, se selecciona la implementación adecuada en base a la lista de parámetros recibidos
- Reglas de sobrecarga de métodos:
 - La lista de argumentos debe ser diferente:
 - Tipos distintos
 - Nº distinto de argumentos
 - Orden de argumentos con tipos diferentes, en diferente orden
 - Deben evitarse las confusiones generadas por el “casting” automático
 - El tipo de retorno, puede ser diferente.
 - No es una circunstancia suficiente para considerar las signatura de los dos métodos distintas

Métodos que utilizan un número variable de argumentos

- Un tipo de sobrecarga típico se produce cuando se pasan a un método un número variable de argumentos:

```
public class Statistics {  
    public float average(int x1, int x2) {}  
    public float average(int x1, int x2, int x3) {}  
    public float average(int x1, int x2, int x3, int x4) {}  
}
```

- Estos tres métodos sobrecargados implementan la misma funcionalidad, pero recibiendo un nº variable de parámetros
- J2SE 5.0 introduce la posibilidad de pasar un número variable de parámetros a un método:

```
public class Statistics {  
    public float average(int... nums) {  
        int sum = 0;  
        for (int x : nums) {  
            sum += x;  
        }  
        return ((float) sum)/nums.length;  
    }  
}
```

Sobrecarga de Constructores

- Cuando se instancia un objeto, pueden invocarse diferentes constructores en función del número y el tipo de argumentos utilizados
 - Puede crearse un usuario indicando nombre, apellidos, salario inicial y fecha de nacimiento o solamente indicando nombre y apellidos

```
public class Employee {  
    private static final double BASE_SALARY = 15000.00;  
    private String name;  
    private double salary;  
    Private Date birthDate;  
  
    public Employee(String name, double salary, Date DoB) {  
        this.name = name;  
        this.salary = salary;  
        this.birthDate = DoB;  
  
    }
```

Sobrecarga de Constructores

```
public Employee(String name, double salary, Date DoB){
    this.name = name;
    this.salary = salary;
    this.birthDate = DoB;
}
public Employee(String name, double salary){
    this(name, salary, null);
}
public Employee(String name, Date DoB){
    this(name, BASE_SALARY, DoB);
}
public Employee(String name, double salary, Date DoB){
    this(name, BASE_SALARY;
}
}
```

Los constructores no se heredan

- Una subclase hereda todos los métodos y atributos de su clase padre, excepto los constructores
- Sin embargo, un constructor de una subclase (Manager) puede invocar a cualquier constructor no privado de su clase padre inmediata.
 - `super ()` → Utilizando el número de argumentos apropiado
 - **Cuando no se realiza ninguna llamada super se invoca al constructor de la clase padre por defecto, es decir, se hace una invocación `super()`, pasando como parámetro 0 argumentos**
 - El constructor invocado en la clase padre debe existir obligatoriamente, si no se generará un error en compilación

Los constructores no se heredan

- Ejemplo:

```
public class Manager extends Employee {  
    private String department;  
    public Manager (String name, double salary, String dept) {  
        super(name, salary);  
        this.department = dept;  
    }  
    public Manager(String dept) { //Este código falla. Employee no tiene ningún  
        //constructor sin argumentos  
        //Si no hay invocaciones super(...) o this (...) c  
        //1ª línea de código, por defecto, se invoca a  
        //super();  
        this.department = dept;  
    }  
}
```

La clase Object

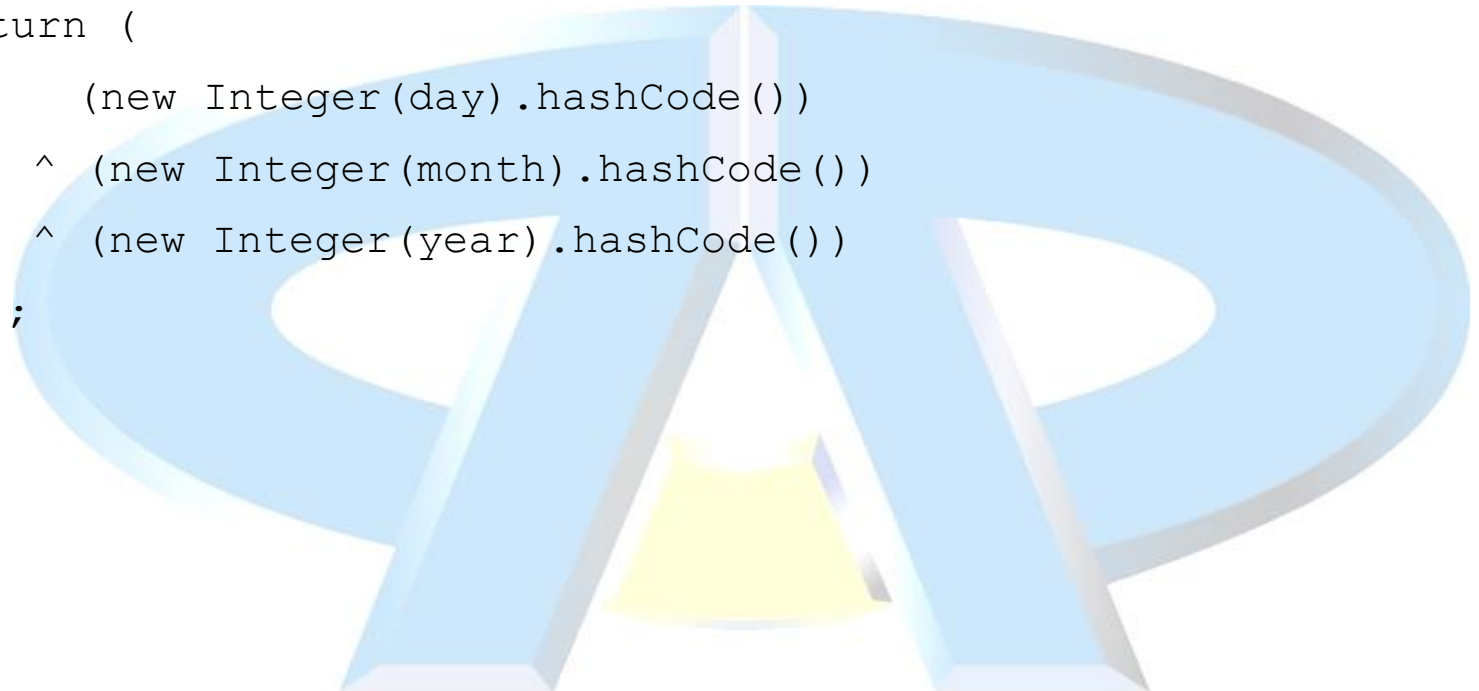
- Es la clase raíz de todas las clases en Java
- Si una clase se declara sin extender a ninguna otra, el compilador añade la cláusula “extends Object”
- Todas las clases heredan los métodos de Object, de entre los que destacan:
 - `public boolean equals(Object obj):`
 - Compara dos objetos
 - Devuelve true si las dos referencias utilizadas apuntan exactamente al mismo objeto
 - Este método es frecuentemente sobreescrito, por ejemplo, en el caso de la clase String
 - `public String toString():`
 - Devuelve una representación de un objeto como una cadena de texto
 - `public int hashCode():`
 - Devuelve un código hash que se utiliza cuando un objeto se utiliza como clave en una estructura hash

La clase Object

```
public class MyDate {  
    private int day, month, year;  
    public MyDate(int day, int month, int year) {  
        this.day    = day;    this.month = month;    this.year  = year;  
    }  
    public boolean equals(Object o) {  
        boolean result = false;  
        if ( (o != null) && (o instanceof MyDate) ) {  
            MyDate d = (MyDate) o;  
            if ( (day == d.day) && (month == d.month)  
                && (year == d.year) ) {  
                result = true;  
            }  
        }  
        return result;  
    }  
}
```

La clase Object

```
public int hashCode() {  
    return (  
        (new Integer(day).hashCode())  
        ^ (new Integer(month).hashCode())  
        ^ (new Integer(year).hashCode())  
    );  
}
```



La clase Object

```
public class TestEquals {  
    public static void main(String[] args) {  
        MyDate  date1 = new MyDate(14, 3, 1976);  
        MyDate  date2 = new MyDate(14, 3, 1976);  
        if ( date1 == date2 ) {  
            System.out.println("date1 is identical to date2");  
        } else {  
            System.out.println("date1 is not identical to date2");  
        }  
        if ( date1.equals(date2) ) {  
            System.out.println("date1 is equal to date2");  
        } else {  
            System.out.println("date1 is not equal to date2");  
        }  
    }  
}
```

La clase Object

```
System.out.println("set date2 = date1;");
date2 = date1;

if ( date1 == date2 ) {
    System.out.println("date1 is identical to date2");
} else {
    System.out.println("date1 is not identical to date2");
}
}
```